



Workspace + *MAPAs distribuídos.*

Em *10min* você entende como seu agente se organiza por dentro — e por que **organização e mapas** são o que permitem ele escalar com você. Sem MAPA, improvisa e vira bagunça em 1 semana. Com MAPA, escala em produção.

A-HA DESTA AULA

A-HA ENTREGUE

"Mapas mantêm ele afiado. Eu audito uma vez por mês."

Tranquilidade estrutural + responsabilidade leve. Aluno PME para de pensar "*vou ter que arrumar onde guardar 50 coisas*" e entende que kit traz estrutura, agente respeita, e ele só faz audit mensal de 5min.

PRE-FLIGHT CHECK

Antes de *apertar play*.

A6 é modular, mas tem dependência leve: você precisa do workspace já provisionado pelo kit (A3). Sem isso, não tem o que mostrar.

- ✓ **Starter Kit aplicado** (A3) — workspace inicial criado, MAPA.md raiz e 4 MAPAs locais populados
- ✓ **Identidade aprofundada** (A5 — recomendado) — agente já tem SOUL+IDENTITY+USER decentes pra decidir bem onde guardar coisa
- ✓ **Telegram conectado** (A1) — demo do Bloco 5 acontece no Telegram em paralelo com IDE/GitHub aberto
- ✓ **10min sem interrupção** — aula é técnica mas não pesada; o aprofundamento dos MAPAs merece atenção

 **Não rodou o Starter Kit?** Volta na A3. Sem workspace inicial provisionado, a A6 fica abstrata — você precisa ver as 4 pastas reais pra entender o conceito.

ESTRUTURA DA AULA EM VÍDEO

O que você vai aprender em *6 blocos*.

Princípio transversal (continuado de A5): **Telegram aberto em paralelo**. Bruno joga coisas pro agente, aluno vê agente organizando arquivos no workspace ao vivo.

 1.5 MIN

Bloco 1 — Por que workspace organizado

Tese: organização do workspace = capacidade de escalar. Sem orientação, agente cria documentos soltos, inventa pasta, cresce desordenado. Solução: **4 pastas raiz + MAPAs distribuídos**. Mapas é o que faz agente "achar as coisas".

🕒 3 MIN

Bloco 2 — Anatomia do workspace canônico

Coração da aula. Visual: árvore do workspace da Amora real (sanitizada). Bruno narra os 4 grupos: `content/` · `memory/` · `skills/` · `archive/`. **Princípio 12 — Mapas distribuídos** introduzido.

🕒 2 MIN

Bloco 3 — 1 pasta = 1 propósito + `memory/` é território do agente

Cada pasta tem 1 função clara — não é "diversos" / "outros" / "notas". **`memory/` é território DO AGENTE** — você não mexe ali. Daily notes em `memory/YYYY-MM-DD.md` (padrão OpenClaw).

🕒 1 MIN

Bloco 4 — Archive: onde joga as coisas pra trás

`archive/YYYY-MM/` flat por mês. **Nunca deletar — sempre arquivar**. Histórico é vantagem competitiva. Recap A2: snapshots automáticos como rede de segurança.

🕒 2 MIN

Bloco 5 — Demo: agente organizando ao vivo

A-ha da aula. Aluno pede *"registra que tive call com Cayo sobre Hub"* no Telegram. Agente decide onde guardar consultando MAPAs — atualiza **3 arquivos diferentes em ~30s**. *"Você falou. Ele organizou."*

🕒 0.5 MIN

Bloco 6 — Recap + amarração A7

Recap dos 4 grupos · MAPAs · 1 pasta = 1 propósito · território do agente · archive · audit. Amarração: *"Workspace é a estante. A7 é sobre como ele LEMBRA do que tá nessa estante."*

Anatomia do workspace *canônico*.

Cada agente OpenClaw nasce com a mesma estrutura. Kit (A3) provisiona. Agente respeitada. Você audita 1 vez por mês. **Você NÃO customiza no day 1.**

Árvore canônica

```

amora-workspace/
├── SOUL.md · IDENTITY.md · AGENTS.md · USER.md ← identidade (recap)
├── MAPA.md ← lista as 4 pastas + onde encontrar cada coisa
├── content/ ← coisas que ele CRIA (posts, drafts, materiais)
│   ├── MAPA.md ← documenta a si mesma (Princípio 12)
├── memory/ ← coisas pra LEMBRAR (decisões, pendências, dail
│   ├── MAPA.md
│   ├── 2026-05-02.md ← daily note SOLTA na raiz (padrão OpenClaw)
│   ├── context/ ← pessoas, negócios, decisões
│   │   ├── projects/ ← 1 arquivo por projeto
├── skills/ ← capacidades modulares (gancho A8)
│   ├── MAPA.md
├── archive/ ← onde joga as coisas pra trás
│   ├── MAPA.md
└── .env ← credenciais (recap kit A3)
    
```

Os 4 grupos · um a um

1

content/ – coisas que ele CRIA pra você

Posts, drafts, materiais, briefings. Quando você pede "*cria post sobre X*", ele salva aqui. Resultado: tudo que ele produz mora num lugar só, fácil de encontrar e de versionar com você.

2

memory/ – coisas pra LEMBRAR

Decisões, pendências, contexto, daily notes do dia. **Território do agente** (Bloco 3). Resultado: persistência real – agente lembra do que rolou ontem e mês passado sem você repetir.

3

skills/ – capacidades modulares

Cada skill é uma habilidade que ele pode invocar (carrossel, análise, audit, etc.). A8 cobre profundo (criar suas próprias). Resultado: agente cresce em capacidade modular sem inflar o prompt principal.

4

archive/ – onde joga as coisas pra trás

Lixo / coisas obsoletas / pendências fechadas / projetos encerrados. Bloco 4 cobre. Resultado: workspace ativo fica limpo, mas histórico continua acessível pra contexto futuro.

Resumo das 4 pastas em uma tabela

PASTA	O QUE MORA AQUI	QUEM MEXE
content/		
memory/		
skills/		
archive/		

PRINCÍPIO 12 · MAPAS DISTRIBUÍDOS

Por que *cada pasta documenta a si mesma.*

Conceito mais novo da aula. Aluno PME provavelmente nunca viu isso. **MAPAs distribuídos** = cada pasta tem um `MAPA.md` local que documenta o que vive ali dentro. Quando agente precisa decidir onde guardar algo, ele lê o MAPA da pasta-pai e encontra rota.

Por que NÃO um TOOLS.md monolítico

Tentação inicial é fazer 1 arquivo central que liste tudo. Quebra rápido por 2 motivos:

- ✓ **Cresce sem controle.** Em 3 meses tem 800 linhas, ninguém atualiza, vira mentira oficial
- ✓ **Lock-in cognitivo.** Agente carrega 800 linhas no contexto pra cada decisão – desperdício de janela e custo
- ✓ **Sem domínio.** Edição de um pedaço afeta tudo – uma mudança bobinha gera conflito ou bagunça noutro lugar

O que MAPAs distribuídos resolvem

- ✓ **Cada parte sabe de si.** `memory/MAPA.md` documenta só `memory/` · `content/MAPA.md` só `content/`
- ✓ **Carrega só o que precisa.** Agente vai salvar coisa em content? Lê só `content/MAPA.md` · economia de janela
- ✓ **Manutenção localizada.** Atualização de uma pasta não afeta as outras
- ✓ **Audit é leitura paralela.** Cada pasta-MAPA é independente – fácil revisar

💡 **Citação do princípio** (skill `wizard-workspace` do kit): *"Sem MAPA, agente improvisa e vira bagunça em 1 semana."* Não é exagero. É padrão observado em workspaces sem mapa.

Estrutura de um MAPA.md típico

Cada MAPA local segue formato simples (kit já gera versão base). Exemplo de

`memory/MAPA.md` :

MAPA – memory/

Território do agente. Aluno NÃO mexe aqui manualmente.

0 que mora aqui

- `YYYY-MM-DD.md` – daily notes soltas na raiz (padrão OpenClaw)
- `context/people/` – 1 arquivo por pessoa relevante
- `context/business/` – 1 arquivo por negócio
- `context/decisoões/YYYY-MM.md` – decisões do mês
- `context/pendências.md` – lista viva de pendências
- `projects/\_index.md` – master de projetos ativos
- `projects/{nome}.md` – 1 arquivo por projeto

Onde guardar coisa nova

- Decisão tomada hoje → `context/decisoões/2026-05.md`
- Pessoa nova → `context/people/{nome}.md`
- Projeto novo → `projects/{nome}.md` + atualiza `\_index.md`
- Daily note → `YYYY-MM-DD.md` na raiz

Regras

- Daily notes ficam SOLTAS na raiz (não em subpasta sessions/)
- Antes de criar pasta nova, verifica se já existe propósito similar
- Pendência fechada vai pra archive/ (não delete)



Você não precisa decorar isso. Kit gera os 4 MAPAs populados. Agente lê e respeita. Você só audita 1x/mês – em 5min você confere se MAPA bate com realidade da pasta.

APROFUNDAMENTO DO BLOCO 3

memory/ é território do *agente*.

Princípio que pode ser contraintuitivo: aluno acha que *"é meu workspace, posso mexer em tudo"*. Pode – mas em `memory/` você atrapalha mais do que ajuda.

Razão: **memory é o cérebro persistente do agente**. Você criar pasta aleatória ali bagunça os MAPAs e quebra busca semântica (cobertura A7).

Anti-padrão clássico

- Aluno cria `memory/notas-importantes/` aleatório porque "preciso de um lugar pra anotar coisa". Resultado: vira lixeira que ninguém olha. Agente não documenta o que não tem MAPA. **Em 2 semanas, ninguém lembra o que tá lá.**

Padrão correto

Quando você precisar de pasta nova, **conversa com o agente**. Ele decide onde encaixa baseado no MAPA da pasta-pai. Se já existe propósito similar (ex: "decisões"), ele reusa. Se é novo, ele cria + atualiza MAPA + commita.



PROMPT · PEDIR PASTA NOVA

COLA NO TELEGRAM

Preciso guardar [TIPO DE COISA] de forma recorrente.

Antes de criar pasta nova:

1. Lê o MAPA das pastas existentes (memory/, content/, etc.)
2. Verifica se já existe propósito similar onde encaixa
3. Se não existir, propõe pasta nova + onde ela vive + atualiza MAPA
4. Me mostra plano antes de criar – eu confirmo ou ajusto

Princípio 11: detecta antes de criar. Não improvisa estrutura.



Se o agente não estiver entendendo, lembre-se de perguntar: "você tem certeza disso?" ou "você quer que eu explique melhor?"

O que mora em memory/ hoje

SUBPASTA	CONTEÚDO	QUEM MEXE
memory/ raiz	YYYY-MM-DD.md	
memory/context/		
memory/projects/	_index.md	



Frase do Bruno: "Você não mexe em memory/. Pergunta a ele onde tá X. Ele lê o MAPA, te responde." Resistência inicial é normal. Em 2 semanas você confia.

APROFUNDAMENTO DO BLOCO 4

Archive — *nunca deletar, sempre arquivar.*

Princípio absoluto. Pode ser contraintuitivo pra leigo ("*se tá obsoleto pra que guardar?*") — mas tem razão prática forte.

Estrutura simples

```
archive/  
├─ 2026-04/  
│   ├── projeto-X-encerrado.md  
│   ├── decisao-revertida.md  
│   └── pendencia-fechada-Y.md  
├─ 2026-05/  
│   └── ...  
└─ MAPA.md
```

Flat por mês. Sem subpastas profundas. Sem taxonomia complexa. Agente arquiva, fica lá, busca semântica acha quando precisa.

O que vai pra archive

- ✓ **Pendência fechada** — sai de `pendencias.md` · vai pra `archive/2026-05/`
- ✓ **Projeto encerrado** — arquivo de `projects/X.md` migra pra archive
- ✓ **Decisão obsoleta ou revertida** — não some, vira contexto histórico
- ✓ **Daily notes muito antigas** — depois de N dias (default 14) saem da raiz, viram archive
- ✓ **Drafts abandonados** — em vez de deletar, arquiva

Por que NÃO deletar

💡 **Agente pode precisar consultar histórico depois pra contexto.** Cenário real: 6 meses depois você pergunta *"a gente já tentou X uma vez?"* · agente busca em archive · acha decisão revertida · responde com data + raciocínio. Sem archive, ele não tem como saber. *Histórico é vantagem competitiva. Não joga fora — arquiva.*

📘 **Recap A2:** snapshots automáticos do OpenClaw garantem recuperação mesmo se algo se perder. Archive é organização ativa · snapshot é rede de segurança passiva. Os dois trabalham juntos.

MANUTENÇÃO

Audit mensal — *5 minutos por mês.*

Workspace é estrutura **viva**. Cresce com uso. Sem manutenção, MAPAs desencontram da realidade · pendências fechadas não migram · daily notes antigas inflam a raiz. Audit mensal mantém afiado.

O que aluno faz no audit

✓ **Pede o report.** *"olha o workspace, vê como tá. Lista o que tá fora dos MAPAs · pendências fechadas que não migraram pro archive · daily notes antigas pra arquivar."*

✓ **Aprova as mudanças.** Agente propõe lista de arquivamento · você confirma em massa

✓ **Atualiza MAPA se preciso.** Se surgiu propósito novo de pasta, MAPA da pasta-pai ganha entrada

✓ **Tempo total: 5 minutos.** Não é tarefa pesada. É escovação de dente.



**PROMPT · AUDIT MENSAL DO
WORKSPACE**

COLA NO TELEGRAM (1X POR
MÊS)

Roda audit do workspace. Olha:

1. Pendências em pendencias.md que tão fechadas – propõe arquivar
2. Projetos em projects/ que tão encerrados – propõe migrar pra archive
3. Daily notes na raiz de memory/ com mais de 14 dias – propõe arquivar
4. Pastas/arquivos que NÃO estão documentados em nenhum MAPA – lista
5. MAPAs que descreveram coisas que não existem mais – lista
6. Decisões em decisoes/ que foram revertidas/obsoletas – propõe arquivar

Me devolve plano em lista. Eu aprovo em massa.

Princípio: archive em vez de delete. Sempre.



Atualizado em 2024 por @miguelsalazar1977 - versão 1.0.0



Resultado do audit: workspace volta pra estado limpo, MAPAs alinhados com realidade, archive crescendo de forma organizada. Tempo investido: 5min/mês. ROI: agente afiado pelos próximos 30 dias.

BLOCO 5 · DEMO A-HA

Agente organizando *ao vivo*.

Demo concreto que entrega o a-ha. No Telegram, aluno manda 1 frase. Agente organiza em < 30 segundos editando 3 arquivos diferentes — consultando MAPAs pra decidir onde cada coisa vive.

Cenário



PROMPT · 1 FRASE, 3 ARQUIVOS EDITADOS

TELEGRAM

registra que tive call com Cayo hoje sobre o launch do Hub. Ele pediu pra revisar VSL até quinta.



O que o agente faz nos próximos 30 segundos

1

Atualiza `memory/context/people/cayo.md`

Adiciona última interação ("call hoje sobre launch do Hub") + data. Mantém histórico relacional do Cayo no perfil dele.

2

Cria entrada em `memory/2026-05-02.md`

Daily note do dia (solta na raiz, padrão OpenClaw). Registra o evento bruto: *"call com Cayo · launch Hub · revisar VSL até quinta"*.

3

Adiciona pendência em `memory/context/pendencias.md`

Item novo: *"revisar VSL do Hub · prazo quinta · pedido pelo Cayo"*. Tag de projeto pra filtrar depois.

Como ele decidiu onde guardar cada coisa

Não foi mágica · não foi prompt complexo. Foi o agente **lendo os MAPAs**:

- ✓ `memory/context/MAPA.md` diz *"última interação por pessoa em `people/{nome}.md`"* → atualiza Cayo
- ✓ `memory/MAPA.md` diz *"daily note solta na raiz como `YYYY-MM-DD.md`"* → cria 2026-05-02.md
- ✓ `memory/context/MAPA.md` diz *"pendências em `pendencias.md` com tag de `projeto`"* → adiciona item

FRASE DO MOMENTO

"Ele decidiu sozinho onde guardar 3 coisas que eu falei numa frase. Eu não pensei nisso uma vez. Os MAPAs guiaram ele."

- ✓ **A-ha emocional:** aluno PME entende que workspace organizado + MAPAs = *delegação real*. Você fala. Ele organiza. Não precisa lembrar onde tá — pergunta a ele.

TRAVOU? ANTES DE PEDIR AJUDA

Top *problemas comuns da A6.*

SINTOMA	CAUSA PROVÁVEL	AÇÃO
		archive/YYYY-MM/



Cheatsheet completo: [workspace-do-seu-agente.md](#) — árvore canônica · MAPAs distribuídos · 1 pasta = 1 propósito · archive em vez de delete · audit mensal · quando criar pasta nova.



Não conseguiu resolver com a tabela acima? Joga screenshot da estrutura no [grupo Telegram do curso](#). openclawzinho responde em < 30 segundos · humanos em < 5 minutos.

FRASE-MÃE DA A6

Pra colar na *geladeira*.

"Workspace organizado = capacidade de escalar. Mapas em cada pasta = agente achando as coisas. Audit mensal = ele afiado pro próximo mês."

4 pastas raiz · MAPAs distribuídos (Princípio 12) · 1 pasta = 1 propósito · memory/ é território do agente · archive em vez de delete · 5min de audit por mês.

Próximo passo: *A7 — Memória + higiene de contexto*

15 minutos · modular · workspace é a estante. A próxima aula é sobre como ele **LEMBRA** do que tá nessa estante quando você fala com ele — e como você evita encher o contexto dele com lixo. 3 tipos de memória (boot · sessão · semântica) · `/new` · `/compact` · *"salve antes de limpar"*.

→ Ir pra Aula 7

TRAVOU? TÁ NO GRUPO DO CURSO

Comunidade Telegram do curso — t.me/cursoopenclaw — suporte 24/7 (openclawzinho bot + Bruno e equipe presente). Dúvida sobre essa aula, problemas de setup, ou quer compartilhar como aplicou no seu negócio: bora trocar lá.

PRÓXIMO NÍVEL: PIXEL AI HUB

Aplicar agente pra empresa toda — equipe, governance, escala? [Pixel AI Hub](#): segundo cérebro corporativo, comunidade de empreendedores, +50 skills enterprise, encontros mensais ao vivo. Vagas mensais limitadas.

[Bruno Okamoto · Pixel Educação](#)